

ARQUITETURA MACRO DO SISTEMA — VISÃO ESTRUTURAL, COMPONENTES DISTRIBUÍDOS E TOPOLOGIA OPERACIONAL — VERSÃO INICIAL

1. Objetivo do artefato

Este documento define:

- arquitetura macro da plataforma;
- visão estrutural distribuída;
- componentes principais;
- fluxos sistêmicos;
- topologia operacional;
- organização dos bounded contexts;
- integração institucional;
- mecanismos de persistência e observabilidade.

Seu objetivo é:

- consolidar a visão arquitetural global;
- estruturar responsabilidades sistêmicas;
- orientar evolução da plataforma;
- sustentar escalabilidade temporal e institucional;
- formalizar o ecossistema distribuído do domínio.

Este documento representa:

- a visão arquitetural corporativa do sistema.

2. Princípios arquiteturais fundamentais

2.1 Arquitetura orientada a eventos

A plataforma:

- é event-driven;
- temporal;

- distribuída;
- auditável.

2.2 Histórico é ativo central

O sistema:

- preserva causalidade;
- suporta replay;
- reconstrói interpretações historicamente.

2.3 O domínio é distribuído semanticamente

Os bounded contexts:

- possuem autonomia;
- comunicam-se por eventos;
- preservam segregação conceitual.

2.4 Observabilidade é estrutural

Toda operação:

- deve ser rastreável;
- explicável;
- auditável.

3. Visão macro da arquitetura

A plataforma é composta por:

1. Camada de Entrada
2. Camada de APIs
3. Camada de Orquestração
4. Bounded Contexts
5. Motor de Cálculo
6. Event Store
7. Camada de Mensageria
8. Camada de Projeções
9. Camada Analítica
10. Camada de Observabilidade
11. Camada de Governança

4. Visão arquitetural em alto nível

Fluxo estrutural principal

Usuários / Sistemas Externos

- API Gateway
- Serviços de Entrada
- Event Bus
- Bounded Contexts
- Motor de Cálculo
- Event Store
- Projeções
- APIs de Consulta
- Dashboards / Auditoria

5. Camada de Entrada

Objetivo

Receber:

- eventos operacionais;
- integrações;
- comandos administrativos;
- sincronizações externas.

Componentes previstos

- API Gateway
- Authentication Layer
- Web Applications
- Mobile Applications
- Integration APIs

- Import Pipelines

Responsabilidades

- autenticação
- autorização
- validação inicial
- roteamento
- rate limiting
- segregação multi-tenant

6. API Gateway

Objetivo

Centralizar:

- entrada institucional;
- autenticação;
- roteamento;
- observabilidade.

Responsabilidades

- JWT/OAuth2
- RBAC/ABAC
- throttling
- tracing
- auditoria
- segregação institucional

7. Camada de Orquestração

Objetivo

Coordenar:

- comandos;
- workflows;
- integrações;
- pipelines distribuídos.

Características

A orquestração:

- não contém lógica de domínio central;
- coordena execução distribuída.

8. Bounded Contexts

Objetivo

Separar:

- responsabilidades semânticas;
- regras;
- causalidade específica.

Contextos previstos

- Jornada
- Banco de Horas
- Homologação
- Auditoria
- Governança Normativa
- Temporalidade
- Inconsistências
- Segurança Institucional
- Observabilidade
- Retenção e Arquivamento

Comunicação

Contextos comunicam-se:

- prioritariamente por eventos.

9. Motor de Cálculo

Objetivo

Executar:

- interpretação temporal;
- cálculo;
- compensações;
- replay;
- reprocessamento.

Características

O motor:

- é determinístico;
- replayável;
- auditável;
- idempotente.

Entradas

- eventos
- regras
- vigências
- compensações
- homologações

Saídas

- interpretações
- projeções
- inconsistências
- eventos derivados

10. Event Bus / Mensageria

Objetivo

Transportar:

- eventos;
- comandos;
- notificações distribuídas.

Requisitos

- ordenação lógica
- retry
- DLQ
- tracing
- idempotência
- particionamento
- replay

11. Event Store

Objetivo

Persistir:

- fatos históricos imutáveis.

Características

- append-only
- replayável
- versionado
- multi-tenant
- auditável
- temporal

O Event Store é

fonte primária da verdade histórica

12. Camada de Projeções

Objetivo

Produzir:

- modelos consultáveis;
- visões operacionais;
- dashboards;
- timelines.

Tipos previstos

- operacionais
- administrativas
- temporais
- auditoriais
- analíticas

Características

Projeções:

- são derivadas;
- reconstruíveis;
- descartáveis.

13. Camada Analítica

Objetivo

Executar:

- analytics;
- BI;
- indicadores;
- análises institucionais.

Características

Consome:

- projeções;
- eventos históricos;
- snapshots analíticos.

14. Camada de Observabilidade

Objetivo

Garantir:

- tracing;
- logs;
- métricas;
- causalidade distribuída.

Componentes previstos

- tracing distribuído
- central de logs
- métricas
- telemetria temporal
- monitoramento de replay

15. Camada de Governança

Objetivo

Administrar:

- regras;
- políticas;
- vigências;
- compliance;
- retenção.

Responsabilidades

- governança normativa

- versionamento
- retenção
- segregação institucional
- cadeia de custódia

16. Camada de Segurança

Objetivo

Garantir:

- autenticação;
- autorização;
- segregação;
- proteção institucional.

Estratégias previstas

- OAuth2
- OpenID Connect
- RBAC
- ABAC
- MFA
- auditoria reforçada

17. Camada de Persistência

Estrutura conceitual

- Event Store
- Read Models
- Snapshots
- Storage Quente
- Storage Morno
- Storage Frio

Estratégia

Persistência:

- preserva causalidade;
- suporta replay;
- mantém histórico integral.

18. Camada Arquivística

Objetivo

Preservar:

- histórico de longa duração;
- dados regulatórios;
- snapshots históricos.

Características

- baixo custo
- alta durabilidade
- restauração sob demanda

19. Estratégia multi-tenant

Regras fundamentais

A arquitetura deve garantir:

- isolamento lógico
- isolamento causal
- isolamento administrativo
- isolamento auditável

Nenhum componente pode

- misturar tenants;
- compartilhar causalidade indevidamente.

20. Estratégia de comunicação

Comunicação síncrona

Usada para:

- comandos imediatos;
- autenticação;
- consultas críticas.

Comunicação assíncrona

Usada para:

- eventos;
- projeções;
- replay;
- integrações distribuídas.

21. Estratégia de consistência

Modelo adotado

Consistência eventual determinística

Características

O sistema admite:

- convergência posterior;
- atualização assíncrona;
- lag operacional controlado.

22. Estratégia de escalabilidade

Escalabilidade horizontal prevista para

- APIs
- mensageria
- projeções
- replay
- analytics

- observabilidade

Estratégias

- particionamento
- paralelismo
- consumo distribuído
- reidratação incremental

23. Estratégia de resiliência

O sistema deve suportar

- falha parcial
- perda de nó
- replay pós-falha
- recuperação incremental
- reidratação completa

Garantindo

- causalidade preservada
- integridade histórica
- retomada segura

24. Estratégia de replay

Replay deve suportar

- replay parcial
- replay total
- replay normativo
- replay auditorial

Características

Replay:

- é capacidade nativa;
- não ferramenta excepcional.

25. Estratégia de versionamento

O sistema deve versionar

- eventos
- regras
- projeções
- APIs
- snapshots
- schemas

26. Estratégia de integração externa

Integrações previstas

- sistemas legados
- diretórios institucionais
- autenticação corporativa
- APIs governamentais
- pipelines externos

Estratégia

Integrações:

- devem passar por camada anti-corrupção.

27. Estratégia de observabilidade

Toda operação deve gerar

- trace_id
- correlation_id
- causation_id
- logs estruturados
- métricas

28. Estratégia de segurança operacional

Operações críticas exigem

- autenticação reforçada
- autorização contextual
- trilha auditável
- segregação funcional

29. Estratégia de deployment

Modelo previsto

- containers
- orquestração
- pipelines CI/CD
- deploy incremental
- rollback seguro

30. Estratégia de ambientes

Ambientes previstos

- desenvolvimento
- homologação
- staging
- produção
- auditoria isolada

31. Estratégia de disaster recovery

O sistema deve permitir

- recuperação temporal
- replay pós-desastre
- restauração incremental
- failover
- reconstrução institucional

32. Estratégia de compliance

A arquitetura suporta

- LGPD
- auditoria externa
- retenção regulatória
- cadeia de custódia
- governança institucional

33. Estratégia de evolução arquitetural

A arquitetura deve permitir

- evolução incremental
- desacoplamento
- substituição controlada
- versionamento progressivo

Sem violar

- causalidade
- temporalidade
- auditabilidade
- invariantes do domínio

34. Relação com Event Sourcing

A arquitetura:

- materializa Event Sourcing;
- preserva histórico;
- suporta replay extremo.

35. Relação com CQRS

Write side:

- preserva causalidade.

Read side:

- otimiza leitura e projeções.

36. Relação com governança institucional

Toda a arquitetura:

- deve permanecer auditável;

- segregada;
- explicável;
- governável institucionalmente.

37. Objetivo arquitetural final

Esta arquitetura busca:

- transformar histórico em núcleo operacional;
- preservar causalidade institucional distribuída;
- sustentar replay auditável em larga escala;
- garantir governança temporal extrema;
- permitir evolução arquitetural segura de longo prazo.